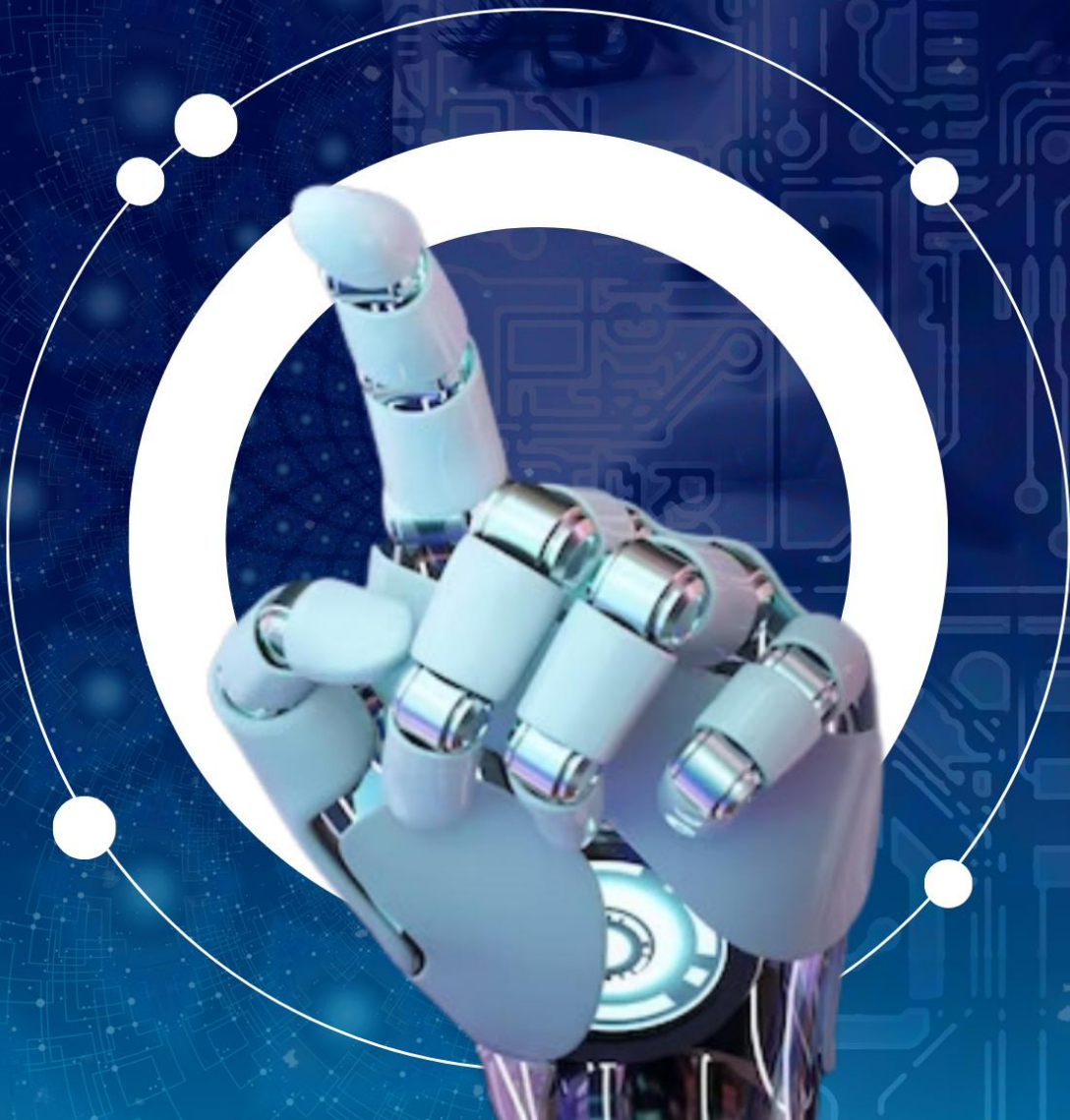


Your paragraph text

A Comparison Between Traditional and AI-Assisted Development Using Student Management System



IT 3003

Name: I.P kaushalya
Index:17353
Reg no:2023s20055

Introduction

AI is becoming an indispensable aspect of modern software development. AI-powered programming tools can help developers by recommending lines of code, detecting bugs, and even as an assistant to explain the specific programming concepts to improve software quality. ChatGPT, GitHub Copilot, AI based debugging assistants are some of such tools.

This report analyzes the role of AI-assisted programming in software development. To examine this topic practically, a Java Student Management System was developed using two different approaches. Version A was developed using a traditional programming approach without AI assistance. Version B was developed using AI assistance to improve selected parts of the program, such as input validation, error handling, and code readability.

The purpose of this report is to compare the two development approaches based on development time, code quality, error handling, readability, and maintainability. The report also discusses the advantages, risks, limitations, and ethical considerations of using AI in programming.

Task 1: Conceptual Understanding

1.1 What is AI-Assisted Programming?

AI assisted programming is utilizing AI tools to assist in software development. Programmers can use these tools in various ways, including code suggestions, error detection, difficult programming concept explanation and code enhancement. For instance, if a programmer encounters an error message they don't know how to handle, they can input it into the AI to get some guidance. For instance, if a programmer is faced with an error message they can't handle, they may type it into the AI and allow it to guide them. AI can also suggest changes to make user input readable, add comments to code, or validate user input. This can help save time, particularly if developers are performing repetitive tasks and/or troubleshooting issues. There are several AI-powered programming assistants available, such as ChatGPT and GitHub Copilot. ChatGPT can respond to programming queries, provide code clarification and offer solutions to errors. While coding within an IDE, GitHub Copilot can provide suggestions for code. AI ought to be employed as a support tool, though, and not as a replacement for the programmer. There is a possibility of error, insecurity, or appropriateness of AI-generated code. Thus, it is crucial for developers to read, comprehend, test, and tweak AI suggestions before incorporating them into their programs.

1.2 Types of AI Coding Tools

There are several types of AI tools that can aid programmers in the software development process. These are helpful for various purposes. There are tools which could generate code from the programmer's instructions. For instance, a developer can request an AI system to generate a Java function whose task is to add a student to the ArrayList. This can save time, particularly in the writing of simple or repetitive sections of programs.

Code completion tools: They guess the next line or block of code as the programmer is typing. GitHub Copilot is one type of such tools. Can aid developers in writing code quickly based on the next needed piece of code.

Debugging Tools: These are used to find and identify errors in a program. An AI can provide an explanation for an error that may be displayed in a program or suggest a potential solution, for instance. Helpful when the students or developer don't know what an error message means.

Refactoring tools are used to enhance the code without altering the functionality of the program. They can possibly offer improved names for variables, eliminate redundant code or

modify methods to be more structured. This can improve the program readability and maintainability. Documentation and explanation tools help explain code in simple language and can be used to add comments and/or documentation. A programmer can instruct AI to give explanations for a Java class or explain the purpose of a method, for instance. Assistance tools can be used for testing to develop test cases and to find possible scenarios for program failures. For instance, an AI tool can recommend experimenting with an option of what the outcome would be when a student enters a letter instead of a number in a mark. These can be useful, but should be checked by the programmer prior to use. In some cases, AI-generated code can be incorrect or not match the requirements of the program.

a. Code Generation Tools

These tools automatically generate source code based on user instructions or descriptions.

Examples:

- GitHub Copilot
- ChatGPT
- Amazon CodeWhisperer

b. Debugging Tools

These tools help identify and fix errors in code by analyzing bugs and suggesting solutions.

Examples:

- ChatGPT
- DeepCode

c. Refactoring Tools

Refactoring tools improve the structure and readability of code without changing its functionality.

Examples:

- IntelliJ IDEA AI Assistant
- Visual Studio AI Tools

d. Code Completion Tools

These tools predict and suggest the next line of code while programming.

Examples:

- GitHub Copilot
- Tabnine

e. Testing Tools

These tools automatically generate test cases and help ensure software quality.

Examples:

- Diffblue Cover
- Testim

1.3 Advantages of AI-Assisted Programming

There are a number of benefits to AI for students and software developers. The primary benefit is it can save time. AI tools can provide suggestions for code, fill in repetitive sections and assist in resolving frequent programming challenges in a faster manner. This frees up time for the developers to concentrate on more important aspects of the system, like designing features, or testing the program. The other is that it is debug friendly. If the program is giving an error message, AI can provide an explanation for the error and solutions. This can be helpful if you're new to programming, as there are some things that can be hard to understand when you first make a mistake. AI has the potential to enhance code quality, as well. They can recommend different names for the variables, improved comments, validate user input, and methods to minimize code repetition. This means the code can be simpler to handle, keep up to date and maintain in the future. AI-powered programming can aid in education, as well. Using AI to ask questions about a programming concept, how a method works, or give examples of a particular topic. This can be helpful to students in grasping the concepts of Java like classes, objects, loops, methods and exception handling. AI can also assist developers in developing documentation and test cases. For instance, it might propose other inputs that could be used, such as illegal ones. It's still the role of the developer to carefully consider the suggestions, however, since AI can occasionally provide wrong or inappropriate responses.

1.4 Risks and Limitations of AI-Assisted Programming

While there are several advantages to AI-assisted programming, there are also some potential risks and limitations to be aware of. Firstly, one of the important points is that AI tools can sometimes generate incorrect code. This code may be correct, but there may be logical errors in it or it may not work when tested. As such, it is not advised for programmers to simply copy AI-generated code without reading and understanding it. Security is another risk factor. AI can propose code which could have security vulnerabilities, particularly when the programmer is not familiar with security concerns. In systems that process sensitive information, like banking systems, healthcare systems, or systems that store personal data, this can be hazardous. AI tools can also cause students to become overly reliant on them. Students should learn to solve problems on their own and program, as otherwise they might rely solely on AI to write the code for them. This can diminish their awareness of key topics like loops, methods, classes and error handling. Another worry is privacy. Developers need to be mindful of giving out sensitive information or code with AI tools, as well as personal details or confidential project information. Do not input sensitive data in AI systems without authorization. Ethical and legal issues can exist, too. AI-generated code could be similar to code from other sources, leading to copyright or licensing problems. Students should also show fairness and honesty in their academic writing by acknowledging the use of AI tools and not claim them as their own. As a result of these considerations, AI should be used with caution as an enabling tool. The final code must be checked, tested and enhanced by the developers before implementing it in the software project.

Task 2: Practical Implementation

Student Management System is one of the selected problems. I chose Java based Student Management System for the implementation. The primary objective of this system is to process and keep the information about students in an easy and orderly manner. It enables the user to Create, Read, Search, Update and Delete the student details. The system holds vital information about individual student like student ID, name, age, and marks. It also computes the mean of all the student's marks in the system. The program will be a console application in which the user will pick the options from a menu and will type the necessary information into the program. I chose this problem since it was appropriate to use object-oriented programming concepts with java. The system utilizes classes, objects, methods, Array List, loops, conditions and user input provided by the Scanner class. It's also a complicated one to compare a traditional programming method to an AI method of programming

Version A: Traditional programming approach (without AI assistance)

Main

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        StudentManager manager = new StudentManager();

        int choice;

        do {
            System.out.println("\n===== STUDENT MANAGEMENT SYSTEM =====");
            System.out.println("1. Add Student");
            System.out.println("2. View Students");
            System.out.println("3. Search Student");
            System.out.println("4. Update Student");
            System.out.println("5. Delete Student");
            System.out.println("6. Calculate Average Marks");
            System.out.println("7. Exit");
            System.out.print("Enter your choice: ");

            choice = scanner.nextInt();
            scanner.nextLine();

            switch (choice) {
                case 1:
                    System.out.print("Enter student ID: ");
                    String id = scanner.nextLine();
```

```

System.out.print("Enter student name: ");
String name = scanner.nextLine();

System.out.print("Enter student age: ");
int age = scanner.nextInt();

System.out.print("Enter student marks: ");
double marks = scanner.nextDouble();
scanner.nextLine();

Student student = new Student(id, name, age, marks);
manager.addStudent(student);
break;

case 2:
System.out.println("\n===== STUDENT LIST =====");
manager.viewStudents();
break;

case 3:
System.out.print("Enter student ID to search: ");
String searchId = scanner.nextLine();
manager.searchStudent(searchId);
break;

case 4:
System.out.print("Enter student ID to update: ");
String updateId = scanner.nextLine();

System.out.print("Enter new name: ");
String newName = scanner.nextLine();

```

```

System.out.print("Enter new age: ");
int newAge = scanner.nextInt();

System.out.print("Enter new marks: ");
double newMarks = scanner.nextDouble();
scanner.nextLine();

manager.updateStudent(updateId, newName, newAge, newMarks);
break;

case 5:
System.out.print("Enter student ID to delete: ");
String deleteId = scanner.nextLine();
manager.deleteStudent(deleteId);
break;

case 6:
manager.calculateAverageMarks();
break;

case 7:
System.out.println("Exiting program...");
break;

default:
System.out.println("Invalid choice. Please try again.");
}

} while (choice != 7);

scanner.close();
}
}

```

StudentManager

```
import java.util.ArrayList;

public class StudentManager {
    private ArrayList<Student> students;

    public StudentManager() {
        students = new ArrayList<>();
    }

    public void addStudent(Student student) {
        students.add(student);
        System.out.println("Student added successfully.");
    }

    public void viewStudents() {
        if (students.isEmpty()) {
            System.out.println("No students found.");
        } else {
            for (Student student : students) {
                System.out.println("-----");
                System.out.println(student);
            }
        }
    }

    public void searchStudent(String studentId) {
        boolean found = false;

        for (Student student : students) {
```

```

if (student.getStudentId().equalsIgnoreCase(studentId)) {
    System.out.println("\nStudent found:");
    System.out.println("-----");
    System.out.println(student);
    found = true;
    break;
}

if (!found) {
    System.out.println("Student not found.");
}

public void updateStudent(String studentId, String newName, int newAge,
double newMarks) {
    boolean found = false;

    for (Student student : students) {
        if (student.getStudentId().equalsIgnoreCase(studentId)) {
            student.setName(newName);
            student.setAge(newAge);
            student.setMarks(newMarks);

            System.out.println("Student updated successfully.");
            found = true;
            break;
        }
    }

    if (!found) {
        System.out.println("Student not found.");
    }
}

```

```

}
}
public void deleteStudent(String studentId) {
    boolean found = false;

    for (Student student : students) {
        if (student.getStudentId().equalsIgnoreCase(studentId)) {
            students.remove(student);
            System.out.println("Student deleted successfully.");
            found = true;
            break;
        }
    }

    if (!found) {
        System.out.println("Student not found.");
    }
}

public void calculateAverageMarks() {
    if (students.isEmpty()) {
        System.out.println("No students found. Cannot calculate average.");
    } else {
        double total = 0;

        for (Student student : students) {
            total = total + student.getMarks();
        }

        double average = total / students.size();
    }
}

```

```
System.out.println("Average marks: " + average);  
}  
}  
}
```

Student

```
public class Student {  
    private String studentId;  
    private String name;  
    private int age;  
    private double marks;  
  
    public Student(String studentId, String name, int age, double marks) {  
        this.studentId = studentId;  
        this.name = name;  
        this.age = age;  
        this.marks = marks;  
    }  
  
    public String getStudentId() {  
        return studentId;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

```

public double getMarks() {
    return marks;
}

```

```

public void setName(String name) {
    this.name = name;
}

```

```

public void setAge(int age) {
    this.age = age;
}

```

```

public void setMarks(double marks) {
    this.marks = marks;
}

```

@Override

```

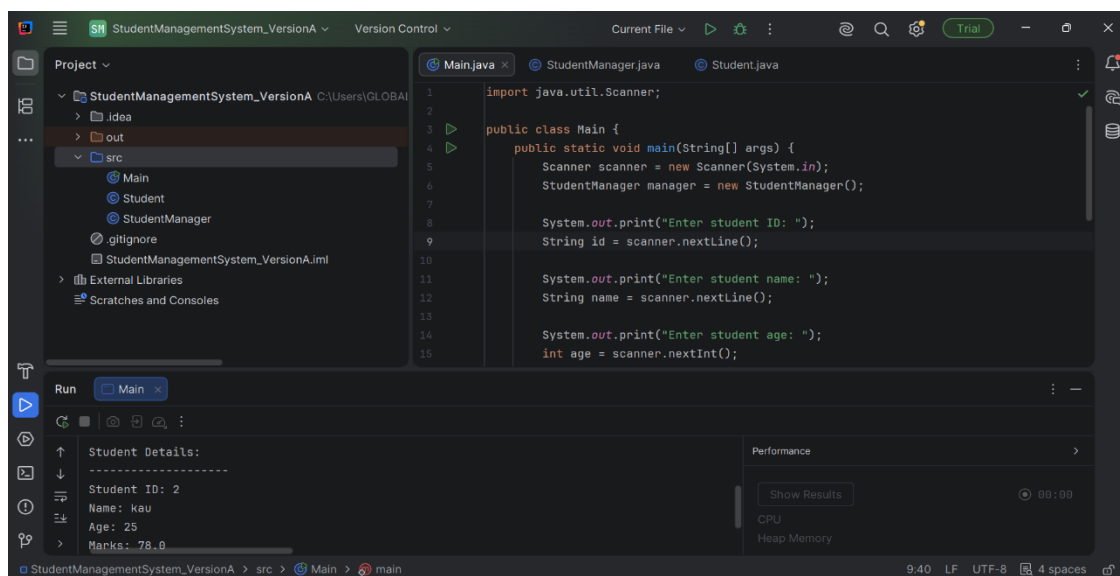
public String toString() {
    return "Student ID: " + studentId
        + "\nName: " + name
        + "\nAge: " + age
        + "\nMarks: " + marks;
}

```

```

}

```



Version B: AI-assisted programming approach using tools such as ChatGPT

Student.Java

```
public class Student {  
    private String studentId;  
    private String name;  
    private int age;  
    private double marks;  
  
    public Student(String studentId, String name, int age, double marks) {  
        this.studentId = studentId;  
        this.name = name;  
        this.age = age;  
        this.marks = marks;  
    }  
  
    public String getStudentId() {  
        return studentId;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public double getMarks() {  
        return marks;  
    }  
}
```

```

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void setMarks(double marks) {
        this.marks = marks;
    }

    @Override
    public String toString() {
        return "Student ID: " + studentId
            + "\nName: " + name
            + "\nAge: " + age
            + "\nMarks: " + marks;
    }
}

```

StudentManager.Java

```
import java.util.ArrayList;

public class StudentManager {
    private ArrayList<Student> students;

    public StudentManager() {
        students = new ArrayList<>();
    }

    public void addStudent(Student student) {
        for (Student existingStudent : students) {
            if
(existingStudent.getStudentId().equalsIgnoreCase(student.getStudentId()))
{
                System.out.println("Student ID already exists. Please use a
different ID.");
                return;
            }
        }

        students.add(student);
        System.out.println("Student added successfully.");
    }

    public void viewStudents() {
        if (students.isEmpty()) {
            System.out.println("No students found.");
        } else {
            for (Student student : students) {
                System.out.println("-----");
                System.out.println(student);
            }
        }
    }
}
```

```

    }
}

public void searchStudent(String studentId) {
    for (Student student : students) {
        if (student.getStudentId().equalsIgnoreCase(studentId)) {
            System.out.println("\nStudent found:");
            System.out.println("-----");
            System.out.println(student);
            return;
        }
    }

    System.out.println("Student not found.");
}

public void updateStudent(String studentId, String newName, int newAge,
double newMarks) {
    for (Student student : students) {
        if (student.getStudentId().equalsIgnoreCase(studentId)) {
            student.setName(newName);
            student.setAge(newAge);
            student.setMarks(newMarks);
            System.out.println("Student updated successfully.");
            return;
        }
    }

    System.out.println("Student not found.");
}

public void deleteStudent(String studentId) {
    for (int i = 0; i < students.size(); i++) {

```

```

        if
(student.get(i).getStudentId().equalsIgnoreCase(studentId)) {
            students.remove(i);
            System.out.println("Student deleted successfully.");
            return;
        }
    }

    System.out.println("Student not found.");
}

public void calculateAverageMarks() {
    if (students.isEmpty()) {
        System.out.println("No students found. Cannot calculate
average.");
        return;
    }

    double total = 0;

    for (Student student : students) {
        total += student.getMarks();
    }

    double average = total / students.size();
    System.out.printf("Average marks: %.2f%n", average);
}
}

```

Main.Java

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        StudentManager manager = new StudentManager();
        int choice;

        do {
            displayMenu();
            choice = readMenuChoice(scanner);

            switch (choice) {
                case 1:
                    addStudent(scanner, manager);
                    break;

                case 2:
                    System.out.println("\n===== STUDENT LIST =====");
                    manager.viewStudents();
                    break;

                case 3:
                    System.out.print("Enter student ID to search: ");
                    manager.searchStudent(scanner.nextLine());
                    break;

                case 4:
                    updateStudent(scanner, manager);
                    break;
            }
        }
    }
}
```

```

        case 5:
            System.out.print("Enter student ID to delete: ");
            manager.deleteStudent(scanner.nextLine());
            break;

        case 6:
            manager.calculateAverageMarks();
            break;

        case 7:
            System.out.println("Exiting program...");
            break;

        default:
            System.out.println("Invalid choice. Please enter a
number from 1 to 7.");
    }

    } while (choice != 7);

    scanner.close();
}

public static void displayMenu() {
    System.out.println("\n===== STUDENT MANAGEMENT SYSTEM =====");
    System.out.println("1. Add Student");
    System.out.println("2. View Students");
    System.out.println("3. Search Student");
    System.out.println("4. Update Student");
    System.out.println("5. Delete Student");
    System.out.println("6. Calculate Average Marks");
    System.out.println("7. Exit");
}

```

```

public static int readMenuChoice(Scanner scanner) {
    while (true) {
        System.out.print("Enter your choice: ");

        if (scanner.hasNextInt()) {
            int choice = scanner.nextInt();
            scanner.nextLine();
            return choice;
        }

        System.out.println("Invalid input. Please enter a number.");
        scanner.nextLine();
    }
}

public static int readValidAge(Scanner scanner) {
    while (true) {
        System.out.print("Enter student age: ");

        if (scanner.hasNextInt()) {
            int age = scanner.nextInt();
            scanner.nextLine();

            if (age > 0 && age <= 120) {
                return age;
            }

            System.out.println("Age must be between 1 and 120.");
        } else {
            System.out.println("Invalid input. Please enter a whole
number.");
            scanner.nextLine();
        }
    }
}

```

```

    }

    public static double readValidMarks(Scanner scanner) {
        while (true) {
            System.out.print("Enter student marks: ");

            if (scanner.hasNextDouble()) {
                double marks = scanner.nextDouble();
                scanner.nextLine();

                if (marks >= 0 && marks <= 100) {
                    return marks;
                }

                System.out.println("Marks must be between 0 and 100.");
            } else {
                System.out.println("Invalid input. Please enter a
number.");
                scanner.nextLine();
            }
        }
    }

    public static void addStudent(Scanner scanner, StudentManager manager)
    {
        System.out.print("Enter student ID: ");
        String id = scanner.nextLine();

        System.out.print("Enter student name: ");
        String name = scanner.nextLine();

        int age = readValidAge(scanner);
        double marks = readValidMarks(scanner);
    }

```

```

        Student student = new Student(id, name, age, marks);
        manager.addStudent(student);
    }

    public static void updateStudent(Scanner scanner, StudentManager
manager) {
        System.out.print("Enter student ID to update: ");
        String id = scanner.nextLine();

        System.out.print("Enter new name: ");
        String newName = scanner.nextLine();

        int newAge = readValidAge(scanner);
        double newMarks = readValidMarks(scanner);

        manager.updateStudent(id, newName, newAge, newMarks);
    }
}

```

AI Prompts

Create a Java Student Management System using three classes: Student, StudentManager, and Main with features to add, view, search, update, delete students, calculate average marks, and exit.

Improve the Student Management System by preventing duplicate student IDs when adding new students.

Add input validation so that age is between 1 and 120 and marks are between 0 and 100.

Comparison of Version A and Version B

Comparison Criteria	Version A: Traditional Programming	Version B: AI-Assisted Programming
Development Time	The system was developed manually. More time was needed to plan the program structure, write methods, and solve errors.	AI suggestions helped create validation and error-handling methods more quickly. However, time was still needed to review and test the suggested code.
Code Quality	The code provides the required functions, including adding, viewing, searching, updating, deleting students, and calculating average marks. However, it has limited validation.	The code provides the same functions with improved validation. It prevents duplicate student IDs and checks valid age and marks ranges.
Error Handling	The program may stop or show an input error when letters are entered where a number is expected.	The program checks numeric input using hasNextInt() and hasNextDouble(). It displays an error message and asks the user to enter a valid value again.
Readability and Maintainability	The code is simple and understandable, but validation is not separated into reusable methods.	The code uses reusable methods such as readValidAge() and readValidMarks(). This makes the code easier to update and maintain.

Table 1

UML Diagrams

Use Case Diagram

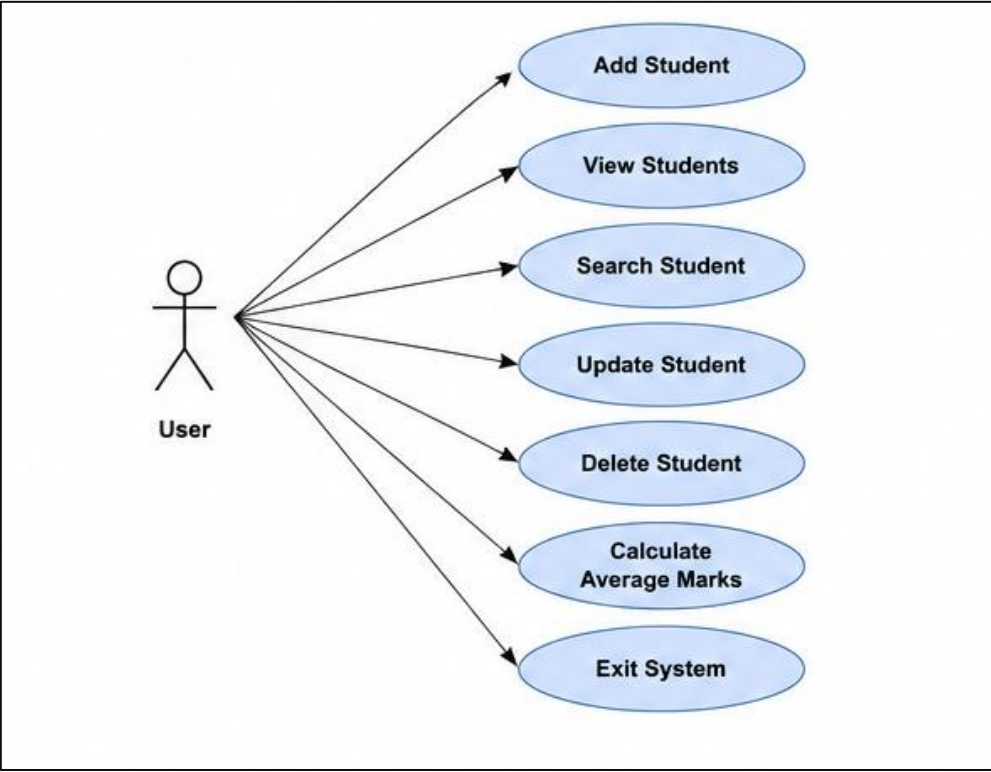


Figure 1

Class Diagram

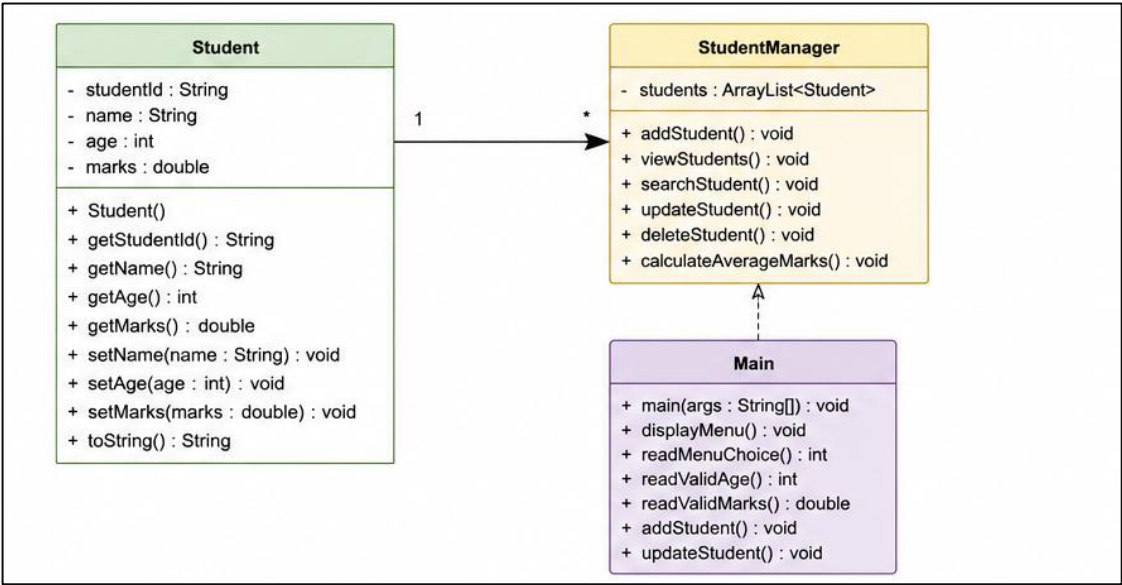


Figure2

Activity Diagram

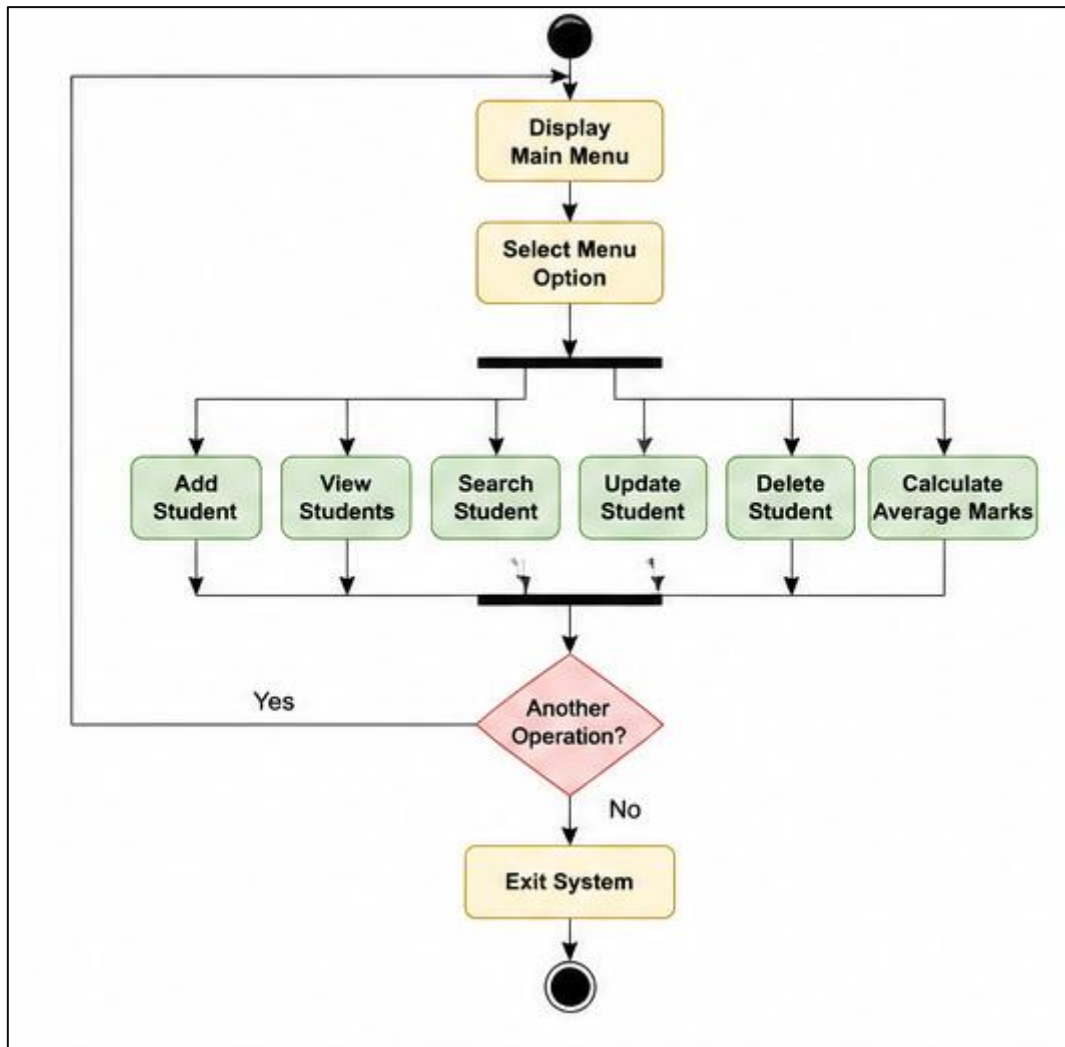


Figure 3

Sequence Diagram

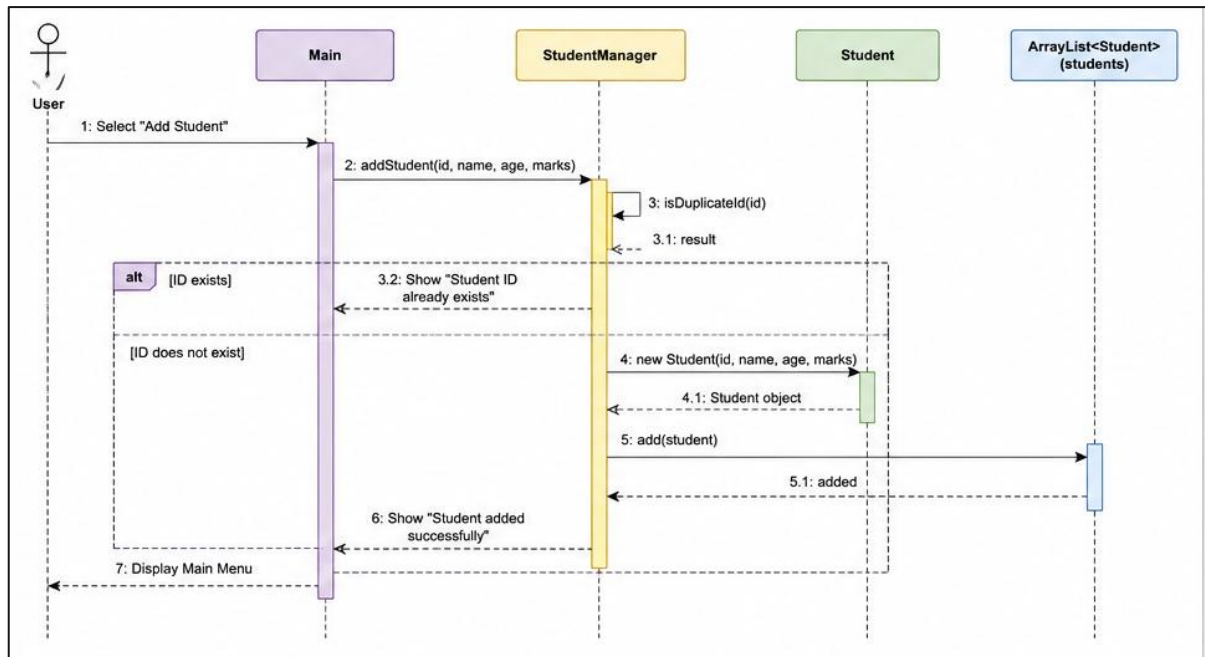


Figure 4

Identification of Manual and AI-Assisted Code

Code Section	Version	Development Method	Description
Project structure (Student.java, StudentManager.java, Main.java)	Version A	Manually Written	The project structure and Java classes were designed and implemented without AI assistance.
Student class (attributes, constructors, getters, setters, toString())	Version A	Manually Written	Stores student information and provides methods to access and update student data.
Menu system (main(), menu display, switch-case)	Version A	Manually Written	Handles user interaction and program navigation through a menu-driven interface.
Student management functions (add, view, search, update, delete, average)	Version A	Manually Written	Implements the core functionality of the Student Management System.
Duplicate Student ID validation	Version B	AI-Assisted	AI suggested checking whether the entered student ID already exists before adding a new student. The code was reviewed, tested, and integrated.
Age and marks validation	Version B	AI-Assisted	AI suggested validating acceptable ranges for age and marks to improve data accuracy.
Invalid input handling (hasNextInt(), hasNextDouble())	Version B	AI-Assisted	AI suggested input validation to prevent runtime errors caused by invalid user input.
Helper methods (readValidAge(), readValidMarks(), displayMenu(), readMenuChoice())	Version B	AI-Assisted	AI suggested reusable validation methods to improve readability and maintainability.
Testing and verification	Version B	Manual Review	All AI-generated suggestions were manually reviewed, tested, and modified before inclusion in the final program.

Table 2

Task 3: Critical Analysis

Impact of AI on the Development Process

The development of the Student Management System using both traditional programming and AI-assisted programming provided valuable insights into the software development process. In Version A, all coding tasks were completed manually, including designing classes, implementing methods, debugging errors, and testing the program. This approach required more time and careful attention to syntax, logic, and program structure.

During the development of Version B, AI was used as a supporting tool to improve the existing program. AI suggested solutions for duplicate student ID validation, input validation, error handling, and code organization. These suggestions reduced the time required to implement additional features and helped improve the overall quality of the program. However, the AI-generated code was not used directly. Each suggestion was carefully reviewed, modified where necessary, and tested before being integrated into the application.

Advantages of Using AI in Programming

The use of AI provided several benefits during software development.

- Reduced the time required to implement new features.
- Suggested alternative solutions for programming problems.
- Improved error handling and input validation.
- Produced cleaner and more organized code.
- Helped understand Java programming concepts through explanations.
- Increased overall development productivity.

Limitations and Challenges

Despite its advantages, AI-assisted programming also has several limitations.

- AI may generate code that does not fully satisfy the program requirements.
- Some generated solutions require modification before they can be used.
- AI does not always understand the complete context of the project.
- Developers still need programming knowledge to identify and correct errors.
- Depending too heavily on AI may reduce opportunities to improve programming skills.

Ethical Considerations

The use of AI in software development also involves ethical responsibilities. Developers should clearly acknowledge when AI tools have been used during development. AI-generated code should always be reviewed, tested, and verified before deployment to ensure reliability and correctness.

Developers must also respect copyright, software licenses, and data privacy when using AI-generated content. AI should support learning and productivity rather than being used to complete assignments without understanding the underlying concepts.

Overall Reflection

This assignment demonstrated that AI can significantly improve software development when used responsibly. Traditional programming helped build a strong understanding of Java programming concepts, while AI-assisted programming improved development efficiency by providing useful suggestions for validation, error handling, and code organization.

The comparison between Version A and Version B shows that AI can enhance software quality and reduce development time. However, human developers remain responsible for understanding requirements, reviewing generated code, testing the software, and making final implementation decisions. Therefore, AI should be considered a powerful development assistant rather than a replacement for software developers.

Task 4: Conclusion

Artificial Intelligence (AI) has become an important part of modern software development and is changing the way programmers design, develop, and maintain software applications. Through this assignment, the role of AI-assisted programming was explored by developing a Java-based Student Management System using two different approaches: traditional programming (Version A) and AI-assisted programming (Version B).

The comparison between the two versions showed that AI can improve software development by reducing development time, improving code quality, enhancing error handling, and increasing code readability and maintainability. AI-assisted programming made it easier to implement features such as duplicate student ID validation, input validation, and improved error handling. However, these AI-generated solutions still required careful review, testing, and modification before they could be integrated into the final application.

This assignment also demonstrated that AI should be used as a supporting tool rather than a replacement for software developers. Programming knowledge is still essential for understanding program requirements, identifying errors, making design decisions, and verifying that AI-generated code functions correctly. Relying entirely on AI without understanding the generated code may lead to incorrect implementations, security issues, or reduced learning opportunities.

From an educational perspective, AI can be a valuable learning resource by explaining programming concepts, suggesting alternative solutions, and helping students debug their programs. In the software industry, AI has the potential to increase developer productivity and improve software quality. However, responsible and ethical use of AI remains essential, particularly in applications where software reliability and security are critical.

Overall, this assignment demonstrates that the combination of traditional programming skills and AI-assisted programming provides the best development approach. AI can significantly improve efficiency and software quality, but successful software development still depends on the knowledge, experience, and critical thinking of human developers.

References

OpenAI. (2026). ChatGPT. <https://chatgpt.com>

Oracle Corporation. (2026). Java Documentation. <https://docs.oracle.com/en/java/>

GitHub, Inc. (2026). GitHub Docs. <https://docs.github.com/>

JetBrains. (2026). IntelliJ IDEA Documentation. <https://www.jetbrains.com/help/idea/>

GitHub repository

<https://github.com/kaushalya-IP/StudentManagementSystem>